

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Jorge Arun Zambrano Cedeño, Mg.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS:

José Luis Solórzano Martínez

PARALELO:

"A"

SEMESTRE:

Quinto

ACTIVIDAD:

N°1

Enlace a Git: <https://github.com/jsolorzano4186/TareasUTM>

PERIODO SEPTIEMBRE 2025 - ENERO 2026



Actividades a desarrollar en la Unidad 1

Actividad # 1

Estudio de caso - Sistema de Gestión de Incidentes (Help Desk)

Objetivo de la tarea:

- Diseñar la arquitectura lógica de un sistema de gestión de incidentes técnicos (Help Desk) para infraestructura de servidores que permita aplicar correctamente patrones de diseño de software y flujos de trabajo profesionales (Gitflow), fortaleciendo la capacidad de análisis, abstracción y buenas prácticas en el desarrollo de soluciones escalables, reutilizables y mantenibles.

Disponibilidad: 20/04/2026 hasta 16/05/2026

Fecha límite: 16/05/2026

Descripción de la Actividad

-  El estudiante deberá diseñar un sistema de Help Desk orientado a la gestión de fallos en un Data Center. El sistema debe permitir reportar incidentes (hardware, red, software), asignar técnicos especializados y gestionar estados de resolución. Para lograr una arquitectura clara y profesional, se deberá utilizar obligatoriamente un repositorio en GitHub bajo el modelo Gitflow y aplicar al menos tres patrones de diseño de software, justificando su elección y uso técnico.

Calificación

-  Tendrá una ponderación de 10 puntos.

Instrucciones

-  Análisis del problema
 - ~ Identificar los actores principales (Usuario final, Técnico de soporte, Administrador de sistemas)
 - ~ Definir los requerimientos funcionales (Crear ticket, categorizar fallo, asignar técnico) y no funcionales (Escalabilidad, mantenibilidad, trazabilidad de versiones).
-  Diseño del sistema
 - ~ Crear un repositorio público en GitHub configurando la estructura de ramas Gitflow (master, develop y ramas feature/ para avances).
 - ~ Crear un diagrama de casos de uso para entender las interacciones del personal de TI con la infraestructura.
 - ~ Realizar un diagrama de clases UML donde se evidencien los objetos y su relación bajo principios SOLID.
 - ~ Diseñar la arquitectura general del sistema integrando el flujo de versiones.
-  Aplicación de patrones de diseño

Utilizar al menos tres patrones. Algunos recomendados:

- ~ Singleton: Para la instancia central del motor de gestión de tickets y conexión a datos.
- ~ Observer: Indispensable para el sistema de notificaciones automáticas a técnicos ante nuevos incidentes (Base de la programación reactiva).
- ~ Factory Method: Para la creación de diferentes tipos de incidentes según el área (Red, Hardware, Software).
- ~ MVC (Model-View-Controller): Como patrón estructural para organizar la lógica de negocio y la interfaz.



- ✚ Documentación del diseño
- ~ Explicar el propósito de cada patrón utilizado y cómo resuelve un problema específico del Help Desk.
- ~ Justificar el uso de Gitflow: Incluir el enlace al repositorio de GitHub y capturas de pantalla del historial de ramas/commits que evidencien el flujo de trabajo.
- ~ Incluir diagramas de clases y secuencia donde se refleje la implementación de los patrones.

Recomendaciones

- ✚ Presentar la tarea con un adecuado formato y presentación de régimen
- ✚ universitario en archivo pdf. Utilizar el formato "Formato_Entrega.docx", estará
- ✚ en la sección de Materiales de aprendizaje / Recursos - Unidad 1 / Recursos
- ✚ Complementarios de la Unidad 1.
- ✚ La actividad deberá ser desarrollada en el tiempo estipulado.
- ✚ La actividad es individual y por lo tanto cada estudiante asumirá la
- ✚ responsabilidad de su autoría.
- ✚ Asegúrate de nombrar y justificar claramente cada patrón.
- ✚ Revisa que los diagramas sean legibles y coherentes con el diseño planteado.
- ✚ Usa nombres significativos para las clases y métodos.
- ✚ No olvides cuidar la presentación del documento: ortografía, redacción clara, numeración de secciones.
- ✚ Puedes usar herramientas como Draw.io, Lucidchart o StarUML para los diagramas.

Diagrama de Casos de Usos

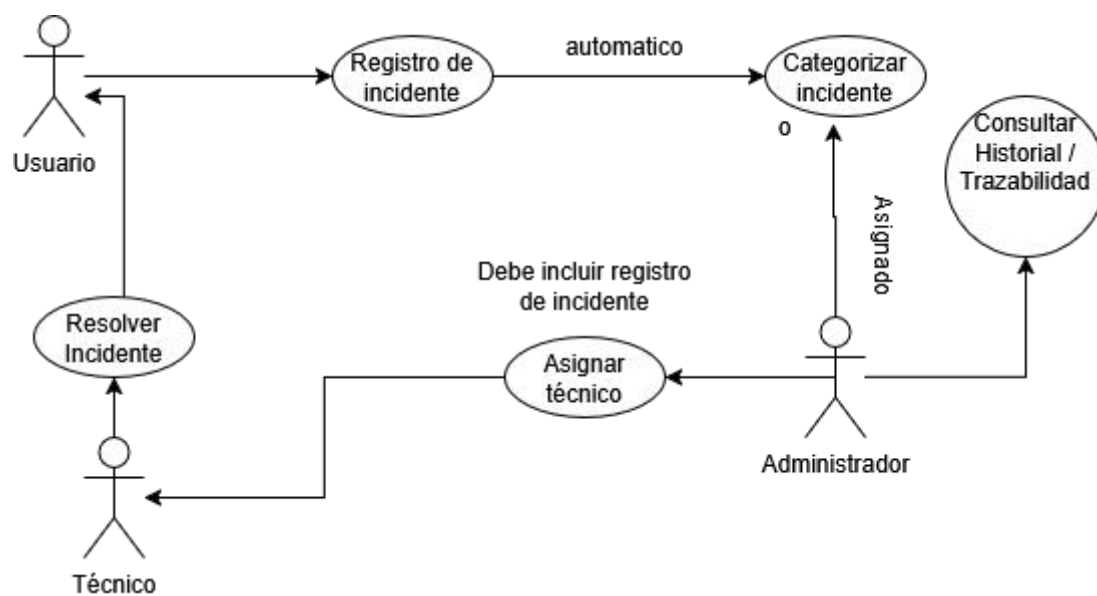
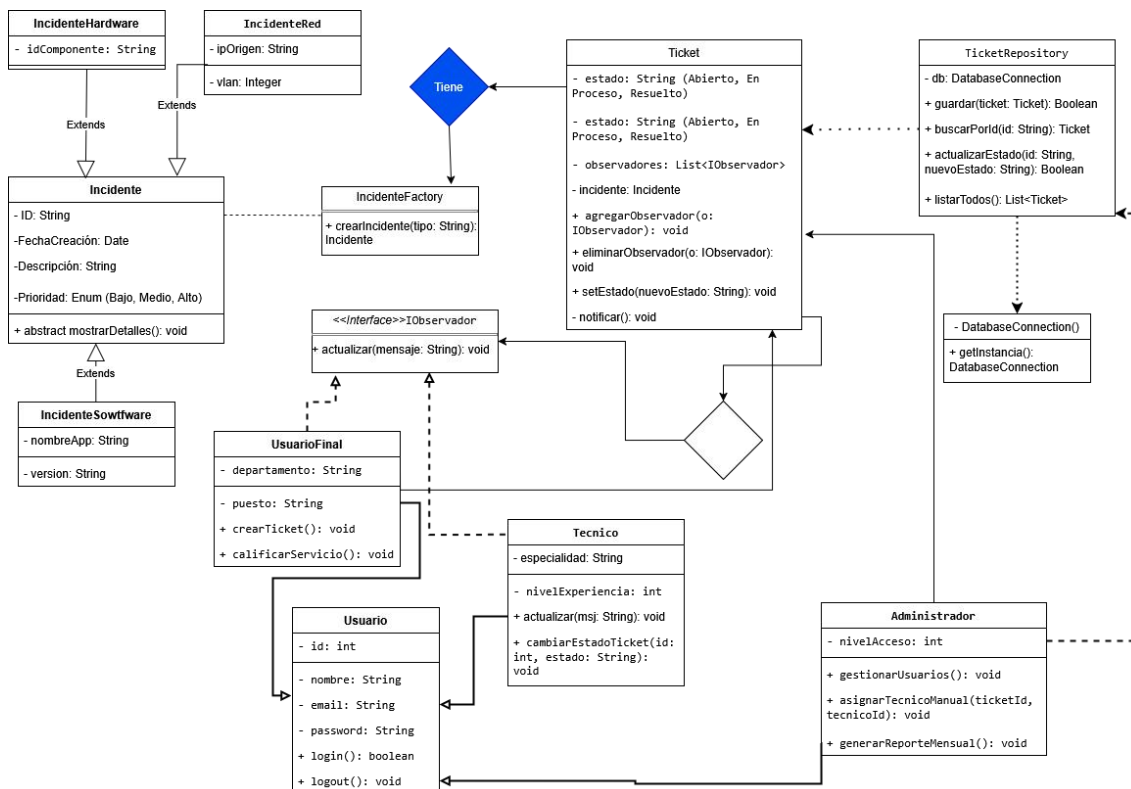


Diagrama de Clases UML



Justificación Técnica de los Patrones

Cuando redactes tu informe, puedes usar estos argumentos técnicos:

Singleton (Conexión a Datos)

"Se implementó para centralizar el acceso a la base de datos del Data Center. En un entorno de servidores, abrir múltiples conexiones puede agotar los recursos del pool. El Singleton garantiza una única instancia global, asegurando la integridad de las transacciones y optimizando el consumo de memoria."

Factory Method (Creación de Incidentes)

"Este patrón resuelve la necesidad de instanciar diferentes tipos de incidentes (Hardware, Red, Software) sin acoplar el código a clases concretas. Facilita la escalabilidad: si en el futuro se requiere gestionar incidentes de 'Ciberseguridad', solo se debe crear la nueva clase y actualizar la fábrica, cumpliendo con el principio Open/Closed de SOLID."

Observer (Notificaciones)

"Esencial para la gestión en tiempo real. Permite que el sistema notifique automáticamente a los técnicos y usuarios cuando el estado de un ticket cambia. Esto elimina la necesidad de consultas manuales constantes (polling), reduciendo la carga del servidor y mejorando el tiempo de respuesta ante fallos críticos."

¿Dónde conectar la clase DatabaseConnection?



En una arquitectura limpia (como MVC o N-Capas), la clase de base de datos **no debe estar regada por todo el código**. Aquí es donde debe conectarse:

A. Con el Patrón Singleton (Definición)

La clase `DatabaseConnection` vive de forma independiente. Su única función es entregar la conexión cuando alguien la pida.

B. Con las clases "Repositorio" o "DAO" (Uso)

Para que tu diagrama UML sea profesional, no conectes la base de datos directamente al `Ticket` o al `Usuario`. Crea una clase intermedia llamada **`TicketRepository`**.

- **La conexión:** El `TicketRepository` llama a `DatabaseConnection.getInstance()` para obtener el permiso de escribir en el disco.
- **En el UML:** Dibuja una flecha de **Dependencia** (línea discontinua) desde el `TicketRepository` hacia `DatabaseConnection.C`. En el flujo del Sistema (MVC)

C. En el flujo del Sistema (MVC)

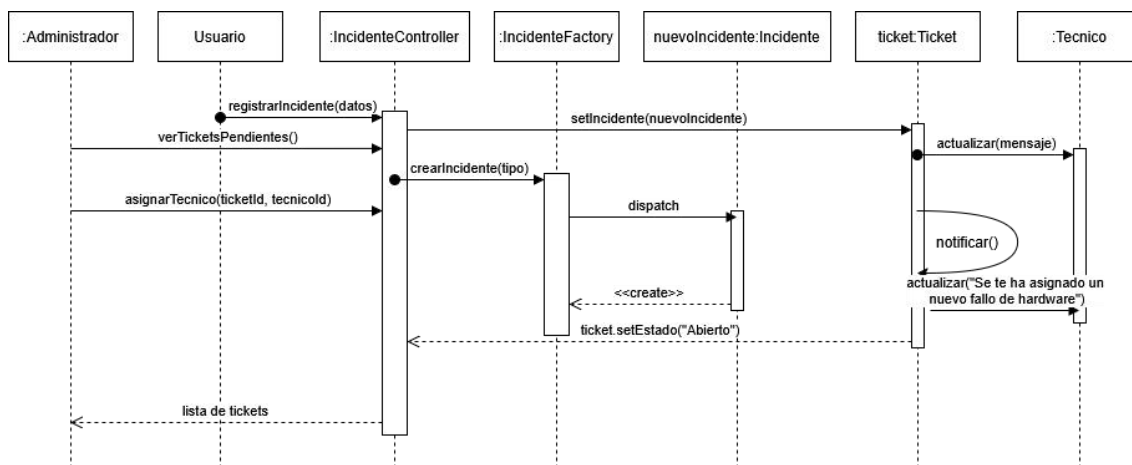
1. El **Controlador** recibe los datos del formulario.
2. El **Controlador** le pide a la **Factory** que cree el objeto `Incidente`.
3. El **Controlador** le pasa ese objeto al **Repositorio**.
4. El **Repositorio** usa el **Singleton** de la base de datos para hacer el `INSERT INTO tickets....`

Resumen de Conexiones Finales para tu UML:

Clase Origen	Clase Destino	Tipo de Relación	¿Por qué?
<code>TicketRepository</code>	<code>DatabaseConnection</code>	Dependencia	Usa el Singleton para guardar datos.
<code>IncidenteFactory</code>	<code>Incidente</code>	Dependencia	Crea las instancias de los hijos.
<code>Ticket</code>	<code>Incidente</code>	Composición	El ticket no tiene sentido sin un incidente.
<code>Ticket</code>	<code>IObservador</code>	Agregación	El ticket avisa a una lista de técnicos/usuarios.



Diagrama de Secuencia



Cuadro de Conexiones: Flujo de Gestión Administrativa

Inicio (Objeto)	Fin (Objeto)	Mensaje / Acción	Tipo de Línea	Explicación Técnica
Administrador	Controller	verTicketsPendientes()	Sólida	El Admin solicita la lista de incidentes sin resolver.
Controller	TicketRepository	obtenerNoAsignados()	Sólida	El Controller pide los datos a la capa de persistencia.
TicketRepository	Database	SELECT * FROM...	Sólida	Uso del Singleton para consultar la base de datos.
Administrador	Controller	asignarTecnico(id, tec)	Sólida	El Admin elige un técnico para un ticket



Inicio (Objeto)	Fin (Objeto)	Mensaje / Acción	Tipo de Línea	Explicación Técnica
				específico.
Controller	Ticket	<code>setEstado("Asignado")</code>	Sólida	Se cambia el estado del objeto en memoria.
Ticket	Ticket	<code>notificar()</code>	Sólida (Bucle)	El Sujeto (Ticket) inicia el ciclo del patrón Observer .
Ticket	Tecnico	<code>actualizar(msj)</code>	Sólida	Se envía la notificación automática al observador.
Controller	Administrador	Confirmación OK	Discontinua	El sistema confirma al Admin que la acción fue exitosa.



Evidencias

(Código de la ejecución de los ejercicios, enlace drive donde se encuentren los mismos y Capturas de pantalla claras y relevantes, donde evidencie el proceso realizado).

The screenshot shows the GitHub 'Create repository' page for the user 'jsolorzano4186'. The interface is in dark mode. At the top, there's a navigation bar with 'Overview', 'Repositories', 'Projects', 'Packages', and 'Stars'. Below this, the user's profile picture (a teal and white cross) and name 'jsolorzano4186' are visible. The main section is titled 'Repository name *' and contains a text input field with 'TareasUTM'. A green checkmark indicates 'TareasUTM is available.' Below the name field, there's a suggestion: 'Great repository names are short and memorable. How about [bug-free-carnival?](#)'. The 'Description' field is empty, with a character count '0 / 350 characters'. The 'Configuration' section is expanded, showing options for 'Choose visibility *' (set to 'Public'), 'Add README' (toggle 'Off'), 'Add .gitignore' (set to 'No .gitignore'), and 'Add license' (set to 'No license'). A green 'Create repository' button is at the bottom right.

jsolorzano4186 doesn't have any public repositories yet.

Owner *
jsolorzano4186

Repository name *
TareasUTM
✓ TareasUTM is available.

Great repository names are short and memorable. How about [bug-free-carnival?](#)

Description
0 / 350 characters

2 Configuration

Choose visibility *
Choose who can see and commit to this repository
Public

Add README
READMEs can be used as longer descriptions. [About READMEs](#)
Off

Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#)
No .gitignore

Add license
Licenses explain how others can use your code. [About licenses](#)
No license

Create repository



Set up in Desktop or HTTPS SSH <https://github.com/jsolorzano4186/TareasUTM.git>

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# TareasUTM" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/jsolorzano4186/TareasUTM.git
git push -u origin main
```

...or push an existing repository from the command line

```
git remote add origin https://github.com/jsolorzano4186/TareasUTM.git
git branch -M main
git push -u origin main
```

MINGW64:/c/Users/José/Desktop/Repositorio UTM

\$ git commoit -m "commit update: subida del archivo adicional tutorial"
git: 'commoit' is not a git command. See 'git --help'.

The most similar command is
 commit

José@Lemanque MINGW64 ~/Desktop/Repositorio UTM (master)
\$ git commit -m "commit update: subida del archivo adicional tutorial"
On branch master
nothing to commit, working tree clean

José@Lemanque MINGW64 ~/Desktop/Repositorio UTM (master)
\$ git remote add origin https://github.com/jsolorzano4186/TareasUTM.git

José@Lemanque MINGW64 ~/Desktop/Repositorio UTM (master)
\$ ^[[200~git branch -M main~
bash: \$'\E[200~git': command not found

José@Lemanque MINGW64 ~/Desktop/Repositorio UTM (master)
\$ git branch -M main

José@Lemanque MINGW64 ~/Desktop/Repositorio UTM (main)
\$ git push -u origin main



Confirm access

 Signed in as @jsolorzano4186

When you are ready, trigger a mailer to verify your identity.

Verify via email

Tip: You are entering [sudo mode](#). After you've performed a sudo-protected action, you'll only be asked to re-authenticate again after a few hours of inactivity.



Authentication Succeeded

You may now close this tab and return to the application.

 jsolorzano4186 commit inicial: subida de la primera version de la presentacion e029bbd · 13 minutes ago 2 Commits

 Actividad1_Solorzano_Jose.pdf commit inicial: subida de la primera version de la presentacion 13 minutes ago

 README





```
MINGW64:/c/Users/José/Desktop/Repositorio UTM
To https://github.com/jsolorzano4186/TareasUTM.git
* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

José@Lemanque MINGW64 ~/Desktop/Repositorio UTM (main)
$

José@Lemanque MINGW64 ~/Desktop/Repositorio UTM (main)
$ git checkout master
error: pathspec 'master' did not match any file(s) known to git

José@Lemanque MINGW64 ~/Desktop/Repositorio UTM (main)
$ git checkout -b tareas
Switched to a new branch 'tareas'

José@Lemanque MINGW64 ~/Desktop/Repositorio UTM (tareas)
$ git checkout -b Act1
Switched to a new branch 'Act1'

José@Lemanque MINGW64 ~/Desktop/Repositorio UTM (Act1)
$ |

MINGW64:/c/Users/José/Desktop/Repositorio UTM

José@Lemanque MINGW64 ~/Desktop/Repositorio UTM (Act1)
$ git push origin act1
fatal: act1 cannot be resolved to branch

José@Lemanque MINGW64 ~/Desktop/Repositorio UTM (Act1)
$ git push origin act1
fatal: act1 cannot be resolved to branch

José@Lemanque MINGW64 ~/Desktop/Repositorio UTM (Act1)
$ git push origin Act1
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'Act1' on GitHub by visiting:
remote:   https://github.com/jsolorzano4186/TareasUTM/pull/new/Act1
remote:
To https://github.com/jsolorzano4186/TareasUTM.git
* [new branch]      Act1 -> Act1

José@Lemanque MINGW64 ~/Desktop/Repositorio UTM (Act1)
$ rm Actividad1_Solorzano_Jose.pdf

José@Lemanque MINGW64 ~/Desktop/Repositorio UTM (Act1)
$
```



Bibliografía

(Referencias bibliográficas y/o enlaces en Internet (normas APA) que se utilizaron para realizar la práctica.)